

Linked List Notes

Singly • Circular • Doubly

0. Why Linked Lists? (Quick Recap)

	Array	Linked List
Size	Fixed at creation (or costly to resize)	Grows/shrinks one node at a time, no resizing needed
Memory	One contiguous block	Scattered nodes connected by pointers
Access arr[i] / node i	$O(1)$ — direct indexing	$O(n)$ — must traverse from the start
Insert/Delete at front	$O(n)$ — must shift everything	$O(1)$ — just relink a pointer

A linked list trades fast random access for fast insertion/deletion — this trade-off answers almost every 'why use a linked list here?' question.

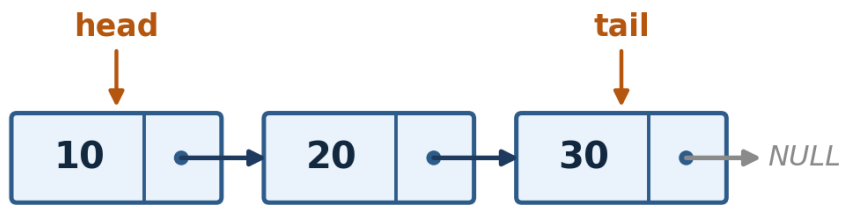
1. Singly Linked List (SLL)

1.1 Node & List Class

The Node is a class holding one value and a pointer to the next node. The list itself is ALSO a class — it privately owns two pointers, head and tail, so both ends of the list are always reachable instantly.

```
class Node {
public:
    int data;
    Node* next;
};

class SinglyLinkedList {
private:
    Node* head;
    Node* tail;
public:
    SinglyLinkedList(){ head = NULL ; tail = NULL;}
    // ... operations declared below are all MEMBER functions of this class
};
```

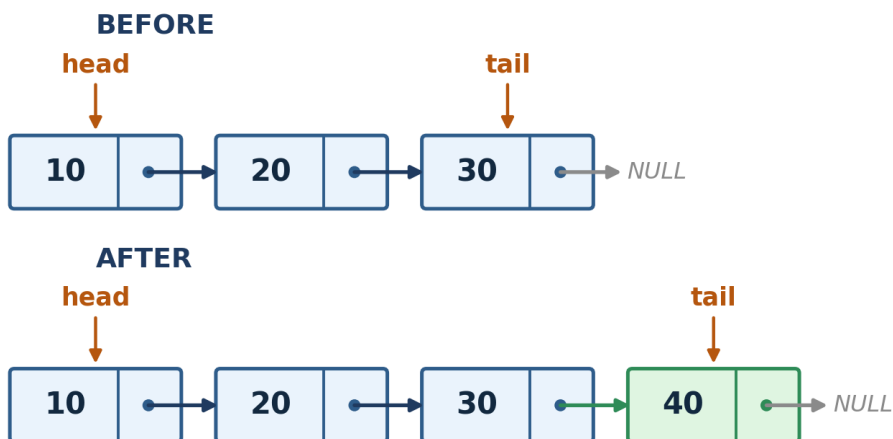
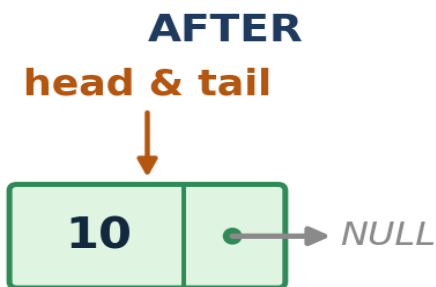


- Keeping tail means insertEnd no longer needs to traverse the whole list to find the last node — it's already known. This turns several $O(n)$ operations into $O(1)$.

1.2 Insertion

(a) Insert into an Empty List, and Insert at End

head = tail = nullptr (empty list)



```

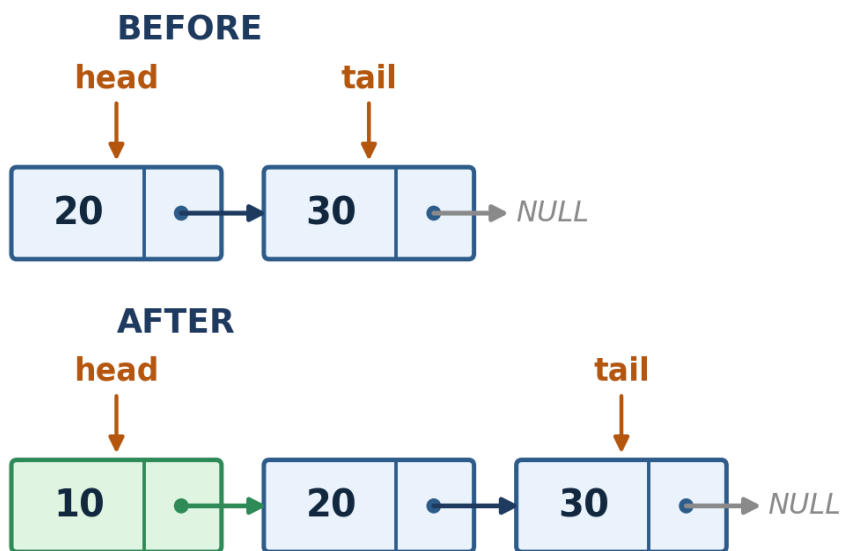
void insertEnd(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = NULL;

    if (head == nullptr) {           // empty list case
        head = tail = newNode;
        return;
    }

    tail->next = newNode;           // O(1) – no traversal needed!
    tail = newNode;
}

```

(b) Insert at the Beginning



```

void insertFront(int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = head;

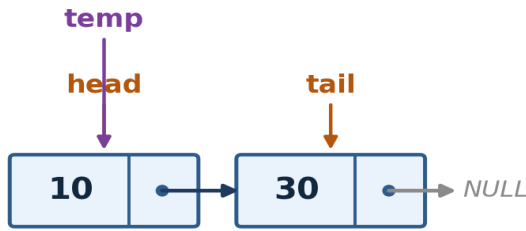
    // checking for empty list
    if (head == nullptr) {
        tail = newNode;
    }

    head = newNode;
}

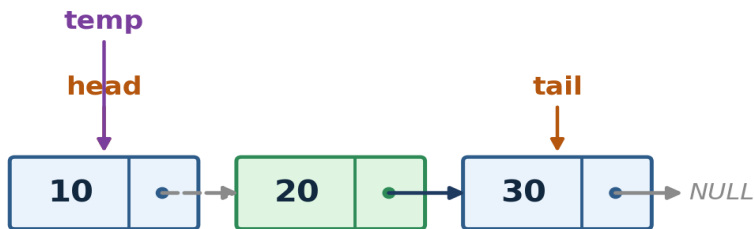
```

(c) Insert at a Specific Position

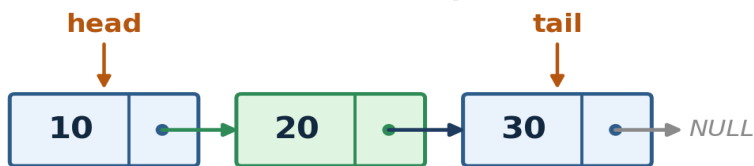
STEP 1: traverse to the node BEFORE position 2



STEP 2: newNode->next = temp->next (not linked in yet)



STEP 3 (AFTER): temp->next = newNode

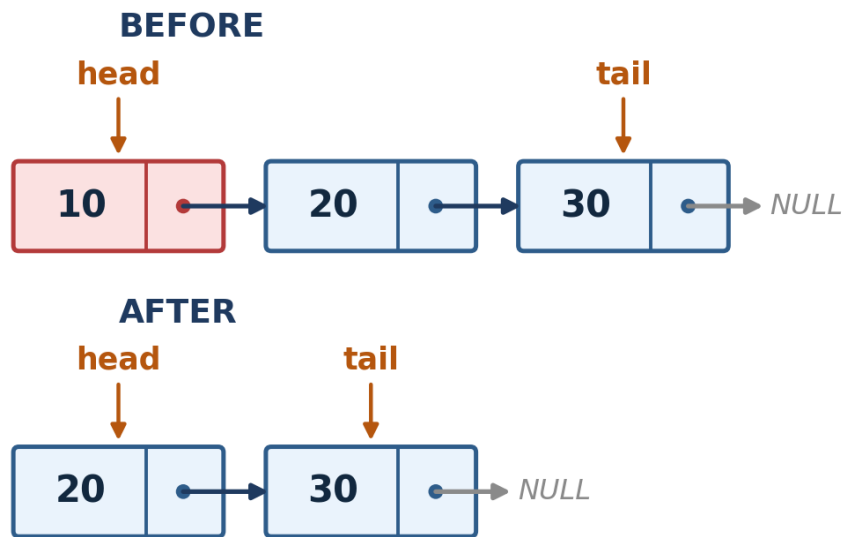


```
void insertAtPosition(int pos, int val) {  
    if (pos == 1) { insertFront(val); return; }  
  
    Node* temp = head;  
    for (int i = 1; i < pos - 1 && temp != nullptr; i++) temp = temp->next;  
  
    if (temp == nullptr) { cout << "Invalid position"; return; }  
  
    Node* newNode = new Node;  
    newNode->data = val;  
    newNode->next = temp->next;           // SAVE the old link first  
  
    temp->next = newNode;                 // then overwrite  
    if (newNode->next == nullptr) tail = newNode; // inserted at the very end  
}
```

- The #1 planted bug: writing 'temp->next = newNode;' BEFORE 'newNode->next = temp->next;' — this loses the rest of the list forever. Always capture the old link first.
- Whenever an insertion could land at the end, ask: 'does tail need to move?' — it's an easy line to forget.

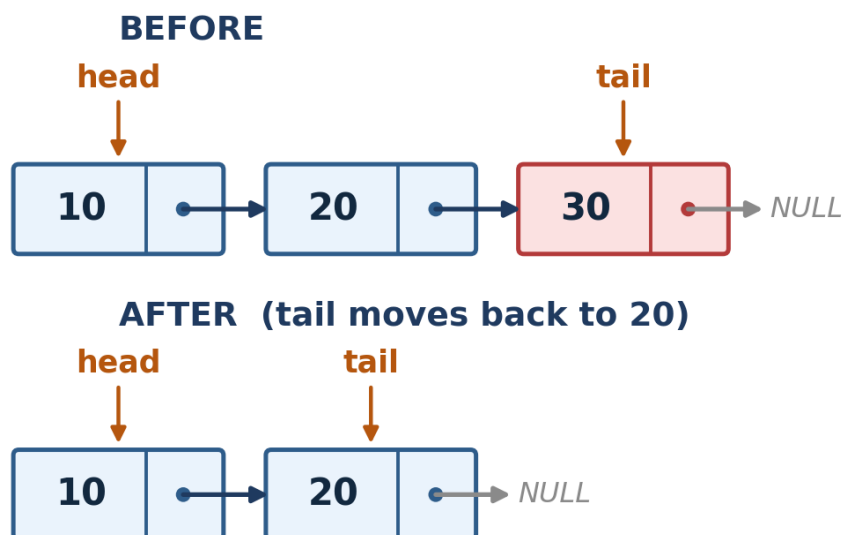
1.3 Deletion

(a) Delete from the Front



```
void deleteFront() {  
    if (head == nullptr) return;           // empty list guard  
    Node* temp = head;  
    head = head->next;  
    if (head == nullptr) tail = nullptr;   // list became empty  
    delete temp;  
}
```

(b) Delete from the End



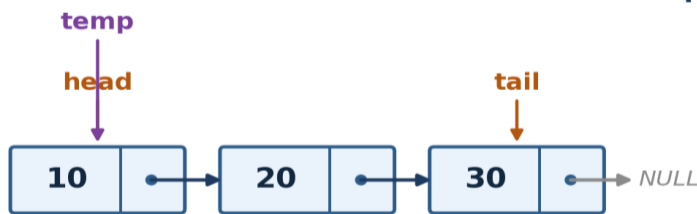
```

void deleteEnd() {
    if (head == nullptr) return;
    if (head == tail) { delete head; head = tail = nullptr; return; } // 1-node case
    Node* temp = head;
    while (temp->next != tail) temp = temp->next;    // still O(n): SLL has no 'prev' link,
                                                    // so we must walk to find the node BEFORE tail
    delete tail;
    tail = temp;
    tail->next = nullptr;
}

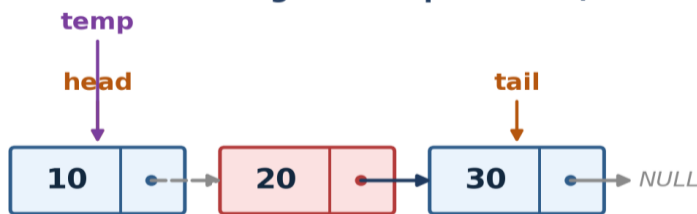
```

(c) Delete at a Specific Position

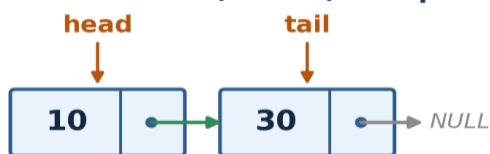
STEP 1: traverse to the node BEFORE position 2



STEP 2: target = temp->next (20 about to be bridged over)



STEP 3 (AFTER): temp->next = target->next, then delete target



```

void deleteAtPosition(int pos) {
    if (head == nullptr) return;
    if (pos == 1) { deleteFront(); return; }
    Node* temp = head;
    //find the node before target
    for (int i = 1; i < pos - 1 && temp->next != nullptr; i++) temp = temp->next;
    if (temp->next == nullptr) { cout << "Invalid position"; return; }
    //cuz we are trying to dlt the node after temp

    Node* target = temp->next;
    temp->next = target->next;    // connection break
    if (target == tail) tail = temp;    // deleted node WAS the tail
    delete target;
}

```

- Always check the empty list (`head == nullptr`) and single-node list (`head == tail`) FIRST — these are the two edge cases every deletion question test.
- Even with tail maintained, `deleteEnd` is still $O(n)$ for a plain SLL — knowing WHERE the last node is, doesn't help you find the node BEFORE it without a 'prev' pointer. That upgrade is exactly what Doubly Linked Lists solve (Section 3).

1.4 Search, Display, Count

```
bool search(int key) {
    Node* temp = head;
    while (temp != nullptr) {
        if (temp->data == key) return true;
        temp = temp->next;
    }
    return false;
}

void display() {
    Node* temp = head;
    while (temp != nullptr) { cout << temp->data << " -> "; temp = temp->next; }
    cout << "NULL" << endl;
}

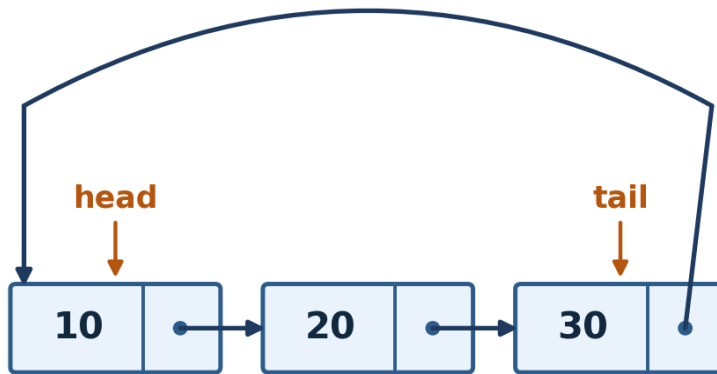
int countNodes() {
    int count = 0;
    Node* temp = head;
    while (temp != nullptr) { count++; temp = temp->next; }
    return count;
}
```

Every SLL traversal follows this exact shape: start temp at head, loop while temp != nullptr, do something, move temp = temp->next. Search, display, and count are the same loop with a different action inside.

2. Circular Linked List (CLL)

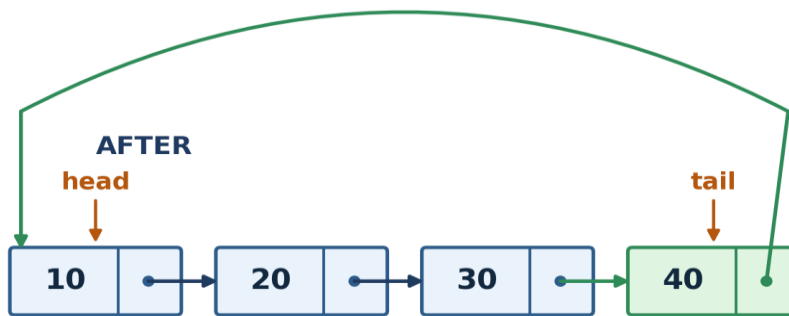
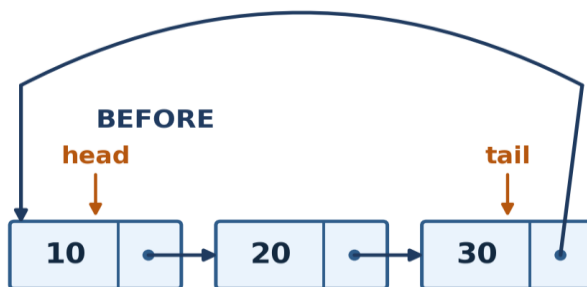
A CLL is an SLL with ONE structural change: the LAST node's next pointer does not become nullptr — it points back to the FIRST node, forming a ring. The class still keeps head AND tail — and here tail is even more valuable, since it lets us reach 'the node right before head' instantly (`tail->next == head`) instead of walking the whole ring to find it.

```
class CircularLinkedList {
private:
    Node* head;
    Node* tail;
public:
    CircularLinkedList(){head = NULL ; tail = NULL;}
};
```



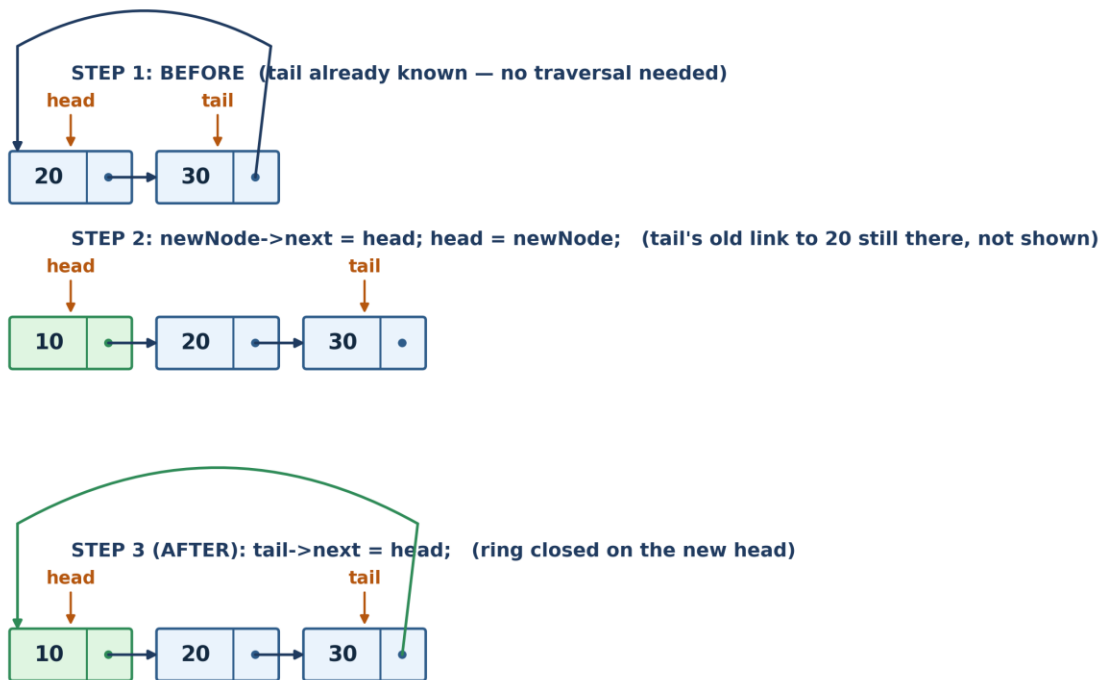
2.1 Insertion

(a) Insert at End (also handles the empty-list case)



```
void insertEnd(int val) {
    Node* newNode = new Node;
    newNode->data = val;
    if (head == nullptr) {
        head = tail = newNode;
        newNode->next = newNode;    // points to itself – a ring of one
        return;
    }
    newNode->next = head;
    tail->next = newNode;          // O(1) – tail already known
    tail = newNode;
}
```

(b) Insert at the Front



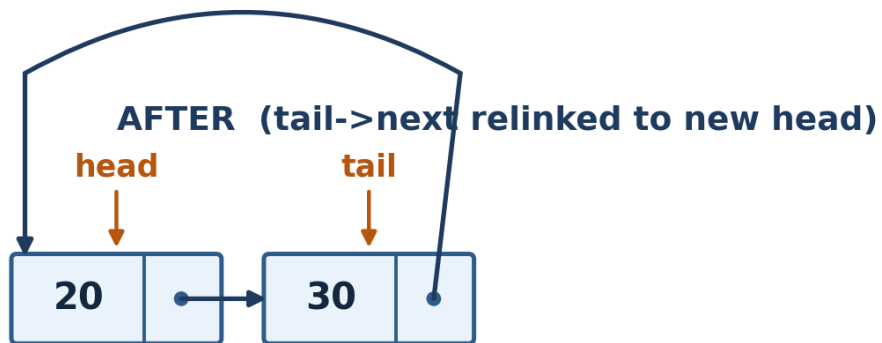
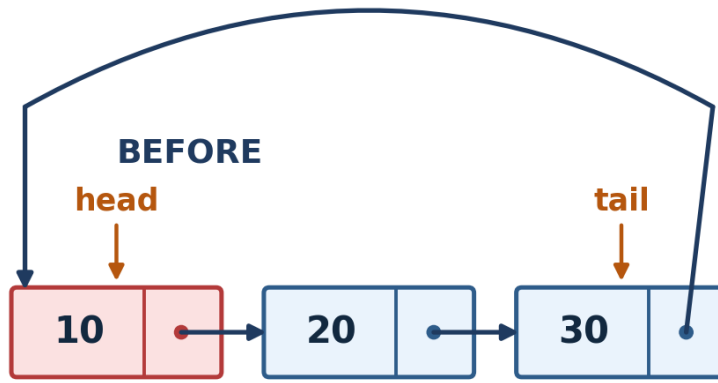
```
void insertFront(int val) {
    Node* newNode = new Node;
    newNode->data = val;
    if (head == nullptr) {
        head = tail = newNode;
        newNode->next = newNode;
        return;
    }
    newNode->next = head;
    head = newNode;
    tail->next = head;           // O(1) – no traversal needed, unlike a head-only CLL
}
```

(c) Insert at a Specific Position

Identical shifting logic to SLL's `insertAtPosition` — traverse to the $(pos-1)$ th node, save-then-relink, and update tail if the new node lands at the end (`temp == tail` before insertion).

2.2 Deletion

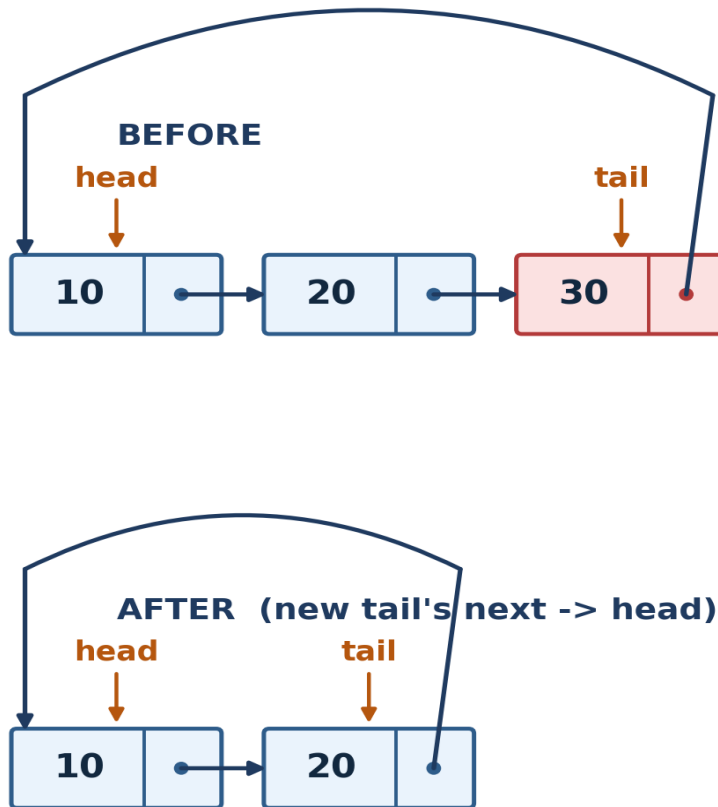
(a) Delete from the Front



```
void deleteFront() {  
    if (head == nullptr) return;  
    if (head == tail) { delete head; head = tail = nullptr; return; }  
    Node* oldHead = head;  
    head = head->next;  
    tail->next = head;  
    delete oldHead;  
}
```

// O(1) – tail already known, no search needed

(b) Delete from the End



```
void deleteEnd() {
    if (head == nullptr) return;
    if (head == tail) { delete head; head = tail = nullptr; return; }
    Node* temp = head;
    while (temp->next != tail) temp = temp->next; // still O(n) - no 'prev' pointer in a CLL

    delete tail;
    tail = temp;
    tail->next = head; // close the ring
}
```

(c) Delete at a Specific Position

Same shifting logic as SLL's deleteAtPosition — update tail if the removed node was the tail.

2.3 Traversal — the Most Important CLL Trap

WRONG: while (temp != nullptr) ... // this NEVER ends – infinite loop!

RIGHT: do {
 cout << temp->data << " ";
 temp = temp->next;
} while (temp != head);

```

void display() {
    if (head == nullptr) { cout << "Empty"; return; }
    Node* temp = head;
    do {
        cout << temp->data << " -> ";
        temp = temp->next;
    } while (temp != head);
    cout << "(back to head)" << endl;
}

```

- If display()/search()/count() uses 'while (temp != nullptr)' on a CLL, it will loop forever — the single most common CLL bug examiners plant.
- Notice how different this section looks from a head-only CLL: because tail is tracked, insertFront/insertEnd/deleteFront are ALL O(1) here — only deleteEnd and position-based operations still need a traversal.

3. Doubly Linked List (DLL)

A DLL node has TWO pointers: one to the next node and one to the previous node. Combined with a head/tail-tracking class, this makes deleteEnd O(1) too — tail->prev gives the second-last node directly, with no traversal at all.

3.1 Node & List Class

```

class Node {
public:
    int data;
    Node* prev;
    Node* next;
};

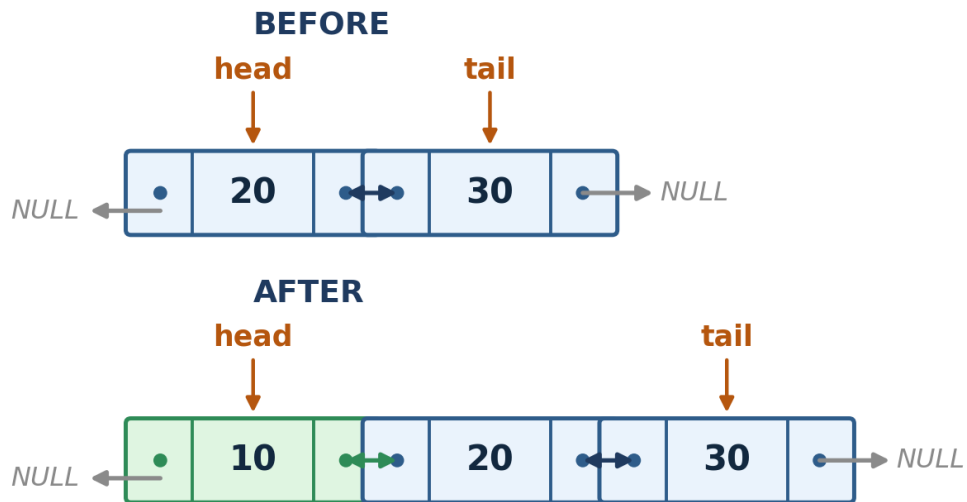
class DoublyLinkedList {
private:
    Node* head;
    Node* tail;
public:
    DoublyLinkedList() {head = NULL ; tail = NULL;}
};

```



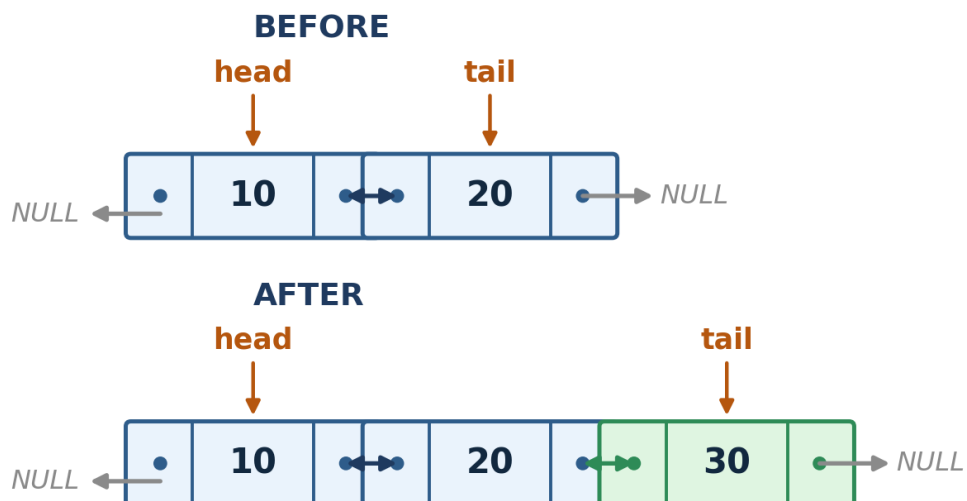
3.2 Insertion

(a) Insert at the Front



```
void insertFront(int val) {  
    Node* newNode = new Node;  
    newNode->data = val;  
    newNode->next = head;  
    if (head != nullptr) head->prev = newNode;  
    else tail = newNode;      // list was empty  
    head = newNode;  
}
```

(b) Insert at the End



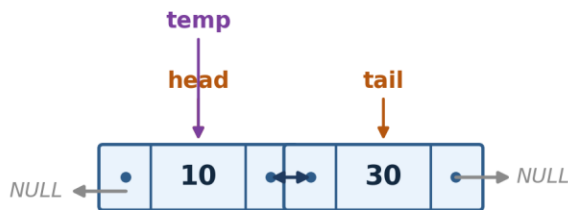
```

void insertEnd(int val) {
    Node* newNode = new Node;
    newNode->data = val;
    newNode->prev = tail;
    if (tail != nullptr) tail->next = newNode;    // O(1) – tail already known
    else head = newNode;                          // list was empty
    tail = newNode;
}

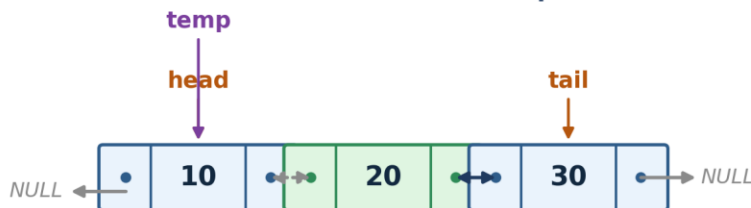
```

(c) Insert at a Specific Position

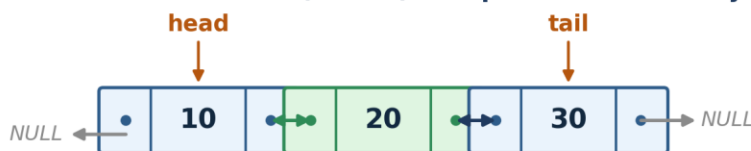
STEP 1: traverse to the node BEFORE position 2



STEP 2: newNode->next/prev set (link FROM temp not made yet)



STEP 3 (AFTER): temp<->newNode fully linked both ways



```

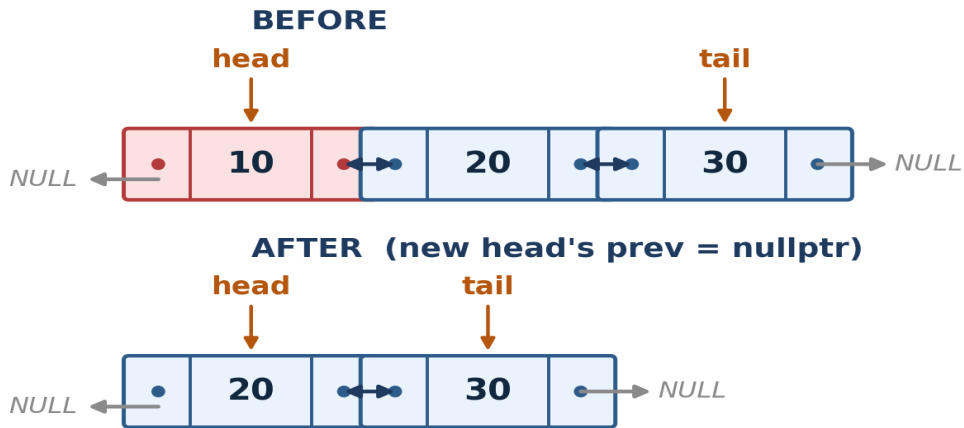
void insertAtPosition(int pos, int val) {
    if (pos == 1) { insertFront(val); return; }
    Node* temp = head;
    for (int i = 1; i < pos - 1 && temp != nullptr; i++) temp = temp->next;
    if (temp == nullptr) { cout << "Invalid position"; return; }
    Node* newNode = new Node;
    newNode->data = val;
    newNode->next = temp->next;
    newNode->prev = temp;
    if (temp->next != nullptr) temp->next->prev = newNode;
    else tail = newNode;          // inserted at the very end
    temp->next = newNode;
}

```

- Every DLL insertion/deletion needs FOUR pointer updates where SLL needed two — for each 'next' relink, ask 'does the matching prev also need updating?'
- Always guard with nullptr checks before dereferencing ->prev or ->next of a possibly-missing neighbor.

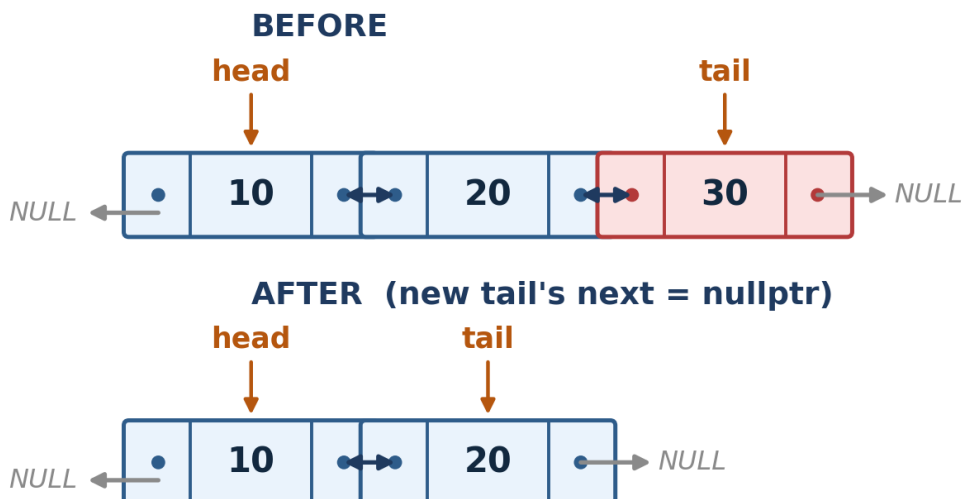
3.3 Deletion

(a) Delete from the Front



```
void deleteFront() {
    if (head == nullptr) return;
    Node* temp = head;
    head = head->next;
    if (head != nullptr) head->prev = nullptr;
    else tail = nullptr;           // list became empty
    delete temp;
}
```

(b) Delete from the End

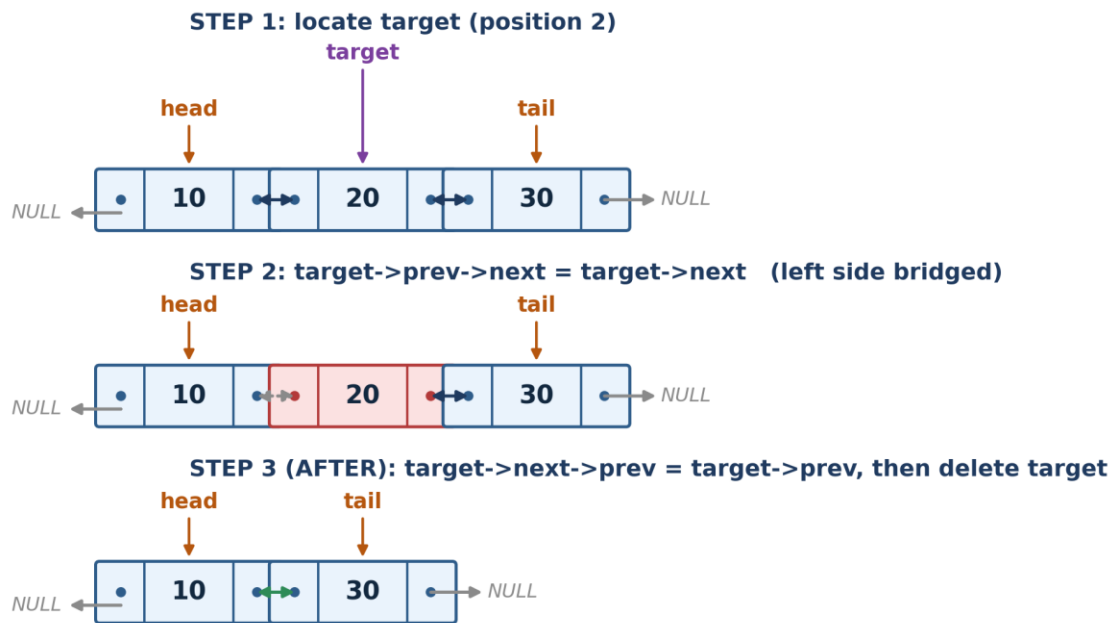


```

void deleteEnd() {
    if (tail == nullptr) return;
    Node* temp = tail;
    tail = tail->prev;           // O(1) - no traversal, unlike SLL!
    if (tail != nullptr) tail->next = nullptr;
    else head = nullptr;        // list became empty
    delete temp;
}

```

(c) Delete at a Specific Position



```

void deleteAtPosition(int pos) {
    if (head == nullptr) return;
    if (pos == 1) { deleteFront(); return; }
    Node* temp = head;
    for (int i = 1; i < pos && temp != nullptr; i++) temp = temp->next;
    if (temp == nullptr) { cout << "Invalid position"; return; }
    if (temp->prev != nullptr) temp->prev->next = temp->next;
    if (temp->next != nullptr) temp->next->prev = temp->prev;
    else tail = temp->prev;           // deleted node WAS the tail
    delete temp;
}

```

Notice DLL deletion never needs to traverse to find 'the node before the target' the way SLL does — `temp->prev` already gives you that node directly. That single extra pointer is the whole advantage.

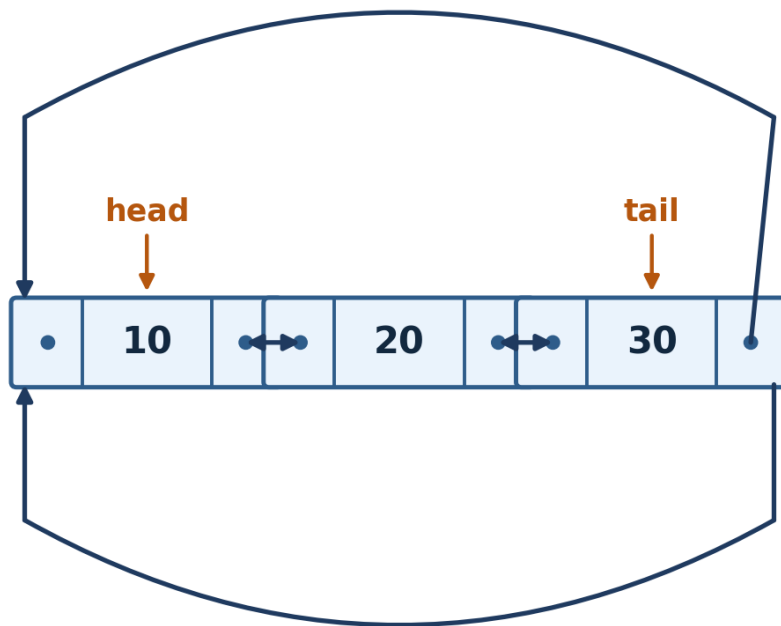
3.4 Traversal — Forward and Backward

```
void displayForward() {
    Node* temp = head;
    while (temp != nullptr) { cout << temp->data << " <-> "; temp = temp->next; }
    cout << "NULL" << endl;
}

void displayBackward() {
    Node* temp = tail;
    // O(1) start point - tail already known
    while (temp != nullptr) { cout << temp->data << " <-> "; temp = temp->prev; }
    cout << "NULL" << endl;
}
```

4. Circular Doubly Linked List (CDLL)

Combine both ideas: prev/next pointers (like DLL) AND the ring closes on both ends (like CLL) — head->prev points to the LAST node (tail), and tail->next points back to head.



- Insertion/deletion logic = DLL's four-pointer relinking + CLL's ring-closing rule (no pointer is ever nullptr).
- Traversal stopping condition = CLL's rule: a do-while loop that stops when you return to head, never 'while (temp != nullptr)'.
- Because both head and tail are tracked as class members, insertFront/insertEnd/deleteFront/deleteEnd are ALL O(1) in a CDLL — the best of every list type combined.
- Real uses: a music player's next/previous with 'loop back to start', or an undo/redo ring buffer.

5. Master Comparison Table

This table assumes every list class maintains BOTH head and tail, as built throughout these notes.

Operation	SLL	CLL	DLL	CDLL
Insert at front	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Insert at end	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$
Delete at front	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Delete at end	$O(n)$	$O(n)$	$O(1)^*$	$O(1)^*$
Traverse backward	Not possible	Not possible	Yes	Yes
Stopping condition	<code>temp == nullptr</code>	<code>temp == head</code> (do-while)	<code>temp == nullptr</code>	<code>temp == head</code> (do-while)
Extra pointer per node	—	—	prev	prev

** Only possible because the class tracks tail. Delete-at-end stays $O(n)$ for SLL/CLL because reaching 'the node before tail' still needs a full traversal without a prev pointer — that gap is exactly what the DLL's prev pointer closes.*

Final exam-day checklist for ANY linked-list question

- 1. Draw the before-state as boxes and arrows on paper first — do not start coding blind.
- 2. Identify which pointers need to change (including head/tail!), and write the SAVE-then-OVERWRITE order before touching code.
- 3. Check the 3 classic edge cases every time: empty list, single-node list, and operation at the very front/back.
- 4. If it's circular (CLL/CDLL): the traversal stopping condition is 'back at head', never nullptr.
- 5. If it's doubly linked: every 'next' relink has a matching 'prev' relink — check both directions.
- 6. Always delete the removed node to avoid a memory leak — but only AFTER re-linking around it.